

Cool Title

DSAN 5500: Data Structures, Objects, and Algorithms in Python

Gentry Lamb

Abstract

In the digital era, data-driven decision-making is paramount, particularly when managing large-scale information flows such as alerts, park activities, and visitor data from the U.S. National Park Service (NPS). However, these data are dispersed across multiple APIs and formats, creating inefficiencies in how updates are retrieved, processed, and utilized. This project presents a robust Extract-Transform-Load (ETL) pipeline built using Python and Prefect that consolidates and streamlines the acquisition, validation, and reporting of NPS data.

The core aim of the pipeline is to support informed trip planning and monitoring of national parks through an automated, reproducible workflow. The pipeline extracts data from the NPS Alerts, Activities, and Parks endpoints, validates and transforms it using Pydantic and Pandera to ensure schema compliance, and loads the cleaned data into structured Markdown reports. These reports can be tailored by state or park, ranked by alert severity, and integrated with user-defined preferences.

By leveraging Prefect for orchestration, the pipeline is modular and observable. Individual tasks are implemented as decorated functions with clear dependencies and Sphinx-style docstrings. The Prefect UI provides visibility into execution, making it easier to troubleshoot failures or analyze runtime. A completion artifact is generated as a Markdown file summarizing the execution duration and the number of parks processed.

This ETL pipeline not only improves efficiency in handling NPS data, but also serves as a reusable framework for integrating other public APIs or expanding user interactivity (e.g., through web dashboards). The current implementation is a Minimum Viable Product (MVP) tailored to the scope of the semester, with a roadmap for scaling to real-time email alerts or a trip planner web app. Through this project, we demonstrate the power of automation, data validation, and orchestration to deliver high-value public information tools.

Introduction & Problem Statement

The current method for gathering information on national parks, such as alerts and activity availability, is largely manual. Users must visit multiple NPS web pages or APIs individually to get up-to-date information. This leads to:

- **Inefficiency:** No centralized process for obtaining alerts or closures by location or user interest.
- **Fragmentation:** Information about parks, activities, and alerts are retrieved separately with inconsistent formats.
- **Lack of validation:** Data pulled from APIs may have missing fields or unexpected formats, risking downstream errors.
- **Poor user experience:** Without integration, it's hard to offer summaries or planning assistance for users.

This ETL pipeline aims to resolve these issues by building a reproducible, automated workflow that collects, cleans, and summarizes data across the three major NPS endpoints. As a Minimum Viable Product, it focuses on:

- Automated retrieval of data via the NPS API.
- Validation of data structure using Pydantic and Pandera.
- Aggregation and ranking of parks by alert severity and content.
- Markdown report generation tailored by state or park.
- A completion artifact summarizing pipeline run details.

ETL Steps Overview

Extract

We use a parameterized `extract_from_api` function to pull JSON from NPS endpoints. Each request uses an API key, and supports queries by state or park code.

Key points: - Rate-limiting is respected via sleep intervals. - Extracted data is immediately passed into transformation steps via Prefect task dependencies.

Validation Point: The raw response is not validated, but is quickly passed into transformation logic which performs schema checks.

Transform

The `transform_alerts_data`, `transform_parks_data`, and `transform_activities_data` functions process the raw data. Here:

- Unwanted fields are dropped.
- Dates are parsed into ISO format.
- Alert severities are classified.
- Pandera is used to validate park and alert schemas to ensure presence of required fields (`parkCode`, `title`, `description`, etc.)

A custom Pydantic model is used to define the structure of park alerts, raising exceptions if required fields are missing or malformed.

Load

The pipeline uses Jinja2 templates to create human-readable Markdown files summarizing: - Park-level or state-level alerts. - Rankings of parks by alert severity. - Optional activities per park.

A final artifact report logs: - Runtime duration - Number of parks processed - Number of alerts ranked

Reports are saved locally as Markdown, but can easily be extended to PDF or emailed via SMTP in future work.

Code & Implementation

Here are sample tasks with docstrings (abbreviated here—full code provided in the appendix):

```
@task
def extract_alerts_data(state: str) -> List[Dict[str, Any]]:
    """
    Extract alert data from the NPS API for a given state.

    Parameters
    -----
    state : str
        Two-letter state abbreviation.

    Returns
    -----
```

```
List[Dict[str, Any]]
    A list of alert dictionaries for parks in the given state.
    """
    url = f"https://developer.nps.gov/api/v1/alerts?stateCode={state}&api_key={API_KEY}"
    response = requests.get(url)
    return response.json()["data"]
```

```
class ParkAlert(BaseModel):
    parkCode: str
    title: str
    description: str
    severity: str
```

```
@task
def transform_alerts_data(alerts: List[Dict[str, Any]]) -> pd.DataFrame:
    """
    Clean and validate alert data.

    Parameters
    -----
    alerts : List[Dict[str, Any]]
        Raw alert data from the API.

    Returns
    -----
    pd.DataFrame
        A cleaned and validated DataFrame of alerts.
    """
    df = pd.DataFrame(alerts)
    df["severity"] = df["severity"].str.lower()
    validated_df = alerts_schema.validate(df)
    return validated_df
```

Prefect UI and Artifacts

Screenshots of Prefect UI: - Successful run flow for `full_pipeline(state='CA')` - Task execution duration for Extract → Transform → Load - Parameters used for a given run

Artifact example:

```
## Pipeline Summary

- **Run Timestamp:** 2025-04-28 12:42 PM
- **State Processed:** CA
- **Total Parks:** 41
- **Total Alerts Retrieved:** 76
- **Pipeline Duration:** 2.3 minutes

Report generated by `plan_your_trip_pipeline`.
```

Conclusion & Future Work

This ETL pipeline demonstrates the effectiveness of combining Python, Prefect, and data validation tools like Pydantic and Pandera to streamline the process of gathering and utilizing public data. The MVP addresses inefficiencies in the manual handling of NPS data by offering automated summaries and recommendations.

Future enhancements could include: - Adding SMTP functionality to email reports. - Deploying the flow on a schedule via Prefect Cloud. - Building a web dashboard for interactive trip planning. - Adding sentiment analysis for user-submitted reviews or alerts.

By focusing on modularity, extensibility, and validation, this project lays the groundwork for a scalable, real-world public data application.